

DNS Watch

By

Baris Ocalan

Submitted to

The University of Roehampton

In partial fulfilment of the requirements

for the degree of

BACHELOR OF SCIENCE/ENGINEERING IN Cyber Security

Declaration

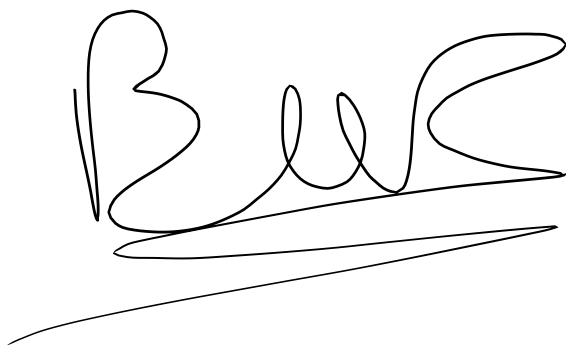
I hereby certify that this report constitutes my own work, that where the language of others is used, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of others.

I declare that this report describes the original work that has not been previously presented for the award of any other degree of any other institution.

Name: Baris Ocalan

Date: 27/04/2026

Signed:

A handwritten signature in black ink, appearing to be 'Baris Ocalan', with a long horizontal line underneath it.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Mastaneh Davis, for her guidance, feedback, and support throughout this project. Her expertise and direction were invaluable in shaping the outcome of DNSWatch. I would also like to thank my friends Sawsan, Adam, Safa, Rhaishah and Faisah for their encouragement and support throughout the year. Finally, I would like to dedicate this work to my mum and my brother, whose continued support made this possible.

Abstract

DNS-based attacks such as tunnelling, domain generation algorithms and fast-flux infrastructure pose a growing threat to the security of computer networks. The solutions that currently exist for mitigating these threats include tools such as SIEM solutions, which require significant infrastructure to be deployed, and tools such as Wireshark, which do not scale to suit the requirements of analysing DNS traffic for signs of malicious activity. DNSWatch was developed to provide a lightweight solution to analysing DNS traffic for malicious activity, using Python to read packet capture files, and automatically detecting suspicious domains.

A literature review was conducted to investigate the different methods for detecting DNS-based threats. The methods that were surveyed included research into lexical analysis, feature analysis, false positive analysis and the use of popularity lists of domains to trust. Some of the authors that were surveyed for the topics and methods included Salat, Davis and Khan on detecting DNS tunnelling, Schüppen et al. on using NXDOMAIN responses for classification of DNS traffic, Holz et al. on detecting fast-flux traffic and Le Pochat et al. on the Tranco list of most popular domains.

DNSWatch was implemented across two modules: an extraction layer using tshark to parse DNS fields from packet captures, and a detection layer applying a seven-feature scoring engine with a multi-signal correlation gate and Tranco-based trust layer. The tool was evaluated against a normal browsing capture and a public dnscat2 DNS tunnelling dataset, producing a false positive rate of 0% and a detection rate of 99.87%. These results demonstrate that a research-backed, transparent and lightweight DNS analysis tool can achieve strong detection performance without enterprise infrastructure.

Table of Contents

Declaration.....	2
Acknowledgements	3
Abstract	4
List of figures	7
List of Tables	7
1. Introduction	8
Problem Statement and/or Research question.....	8
Aims	9
Objectives	9
2. Background and Context.....	10
Background	10
Summary of relevant published work (Literature review)	11
2.2.1 DNS as an attack vector	11
2.2.2 DNS threat detection methods	11
2.2.3 Challenges in DNS detection and false positives	12
2.2.4 Reputation and popularity-based trust approaches	13
2.2.5 Gaps in existing work and relevance to this study	14
Competitor Analysis/ Related work	15
3. Tools and Methods	17
Rationale for the technical tools selected	17
Rationale for the research methods used	18
Project management and Product lifecycle	19
Risk register	20
4. Requirements/Specifications	22
Persona:	22
Functional requirements:	23
Non-Functional requirements:.....	25
Use Case Diagram:	26
5. Solution design	26
Workflow diagram:.....	26
6. Solution implementation	28
Sprint 1 – DNS Extraction Layer (main.py):.....	28

Sprint 2 – Detection and Scoring Engine (detect.py):	29
DNSWatch v2 Scoring Specification	31
Dashboard Output:	32
7. Solution testing/ evaluation	34
Test 1 – Normal browsing capture (false positive rate):	34
Test 2 – dnscat2 DNS Tunnelling Capture (Detection Rate):	34
DNSWatch evaluation results against standard detection metrics	35
Strengths:.....	35
Weaknesses / Limitations:	36
8. Discussion	37
9. Conclusion	38
Summary	38
Recommendations for further work	38
Reflection	39
10. References	40
11. Appendices	42
Appendix A – GitHub repository and access description:.....	42
Appendix B — Evidence of Project Management and Supervision.....	42
Appendix C – Ethics Form	43
Appendix D – Risk Register	44
Appendix E – Professional and Peer Support Badges.....	46

List of figures

Figure 1 Use Case Diagram.....	26
Figure 2 Workflow Diagram.....	27
Figure 3 DNS packet parsing	29
Figure 4 Group by Per Unique Domain	30
Figure 5 Normal traffic dashboard	32
Figure 6 Dashboard for Malicious traffic	33
Figure 7 Mastaneh Catch up Example #1.....	42
Figure 8 Mastaneh Catch up Example #2.....	43
Figure 9 Mastaneh Catch up Example #3.....	43
Figure 10 Ethics form Approval	44
Figure 11 Professional and Peer Supporter Badges	46

List of Tables

Table 1Risk Register	20
Table 2 Functional Requirements	23
Table 3 Non-Functional requirements.....	25
Table 4 DNSWatch v2 Scoring Specification	31
Table 5 DNSWatch evaluation results.....	35
Table 6 Risk Register Appendix D	44

1.Introduction

This study introduces DNSWatch, DNSWatch is a Python-based tool that reads packet capture files, extracts DNS traffic from those files, and scores each domain for suspicious activity using its seven-feature scoring system. DNSWatch was built to be accessible to individuals without enterprise network infrastructure, providing them with an easy-to-understand output of the scored domains and a visual dashboard displaying the analysis of the DNS traffic files. The Domain Name System (DNS) is responsible for translating human-readable domain names into the numerical IP addresses that devices use to communicate across the internet. Because DNS is essential to virtually all network activity, it is permitted through most firewalls and security controls without restriction. This makes it an attractive target for attackers, who exploit DNS traffic to carry out malicious operations while avoiding detection. Common techniques include domain generation algorithms (DGA), which create disposable contact points for malware, DNS tunnelling, which encodes stolen data within DNS queries to secretly move it out of a network, and fast-flux hosting, which rapidly rotates server addresses to avoid being blocked.

Problem Statement and/or Research question

DNS traffic is a critical attack vector which exploits through techniques such as domain generation algorithms, DNS tunnelling, and fast-flux infrastructure. Identifying these threats requires the analysis of DNS query patterns, but the existing solutions fall into two extremes. Enterprise SIEM platforms such as Graylog, Splunk, and Elastic Security offer comprehensive real-time monitoring but require significant infrastructure, licensing costs, policy configuration, and specialist knowledge to deploy making it a pain for the reach of small businesses and individual analysts. At the other extreme, manual analysis using tools like Wireshark is free but time-consuming, error-prone, and does not scale. DNSWatch addresses this gap, it is a lightweight Python-based tool that reads standard packet capture files, automatically detects suspicious DNS activity, and presents the results in a clear scored format that any analyst can interpret without specialist training or enterprise infrastructure.

Aims

- To develop an automated DNS traffic analysis tool capable of detecting malicious domains and anomalies from packet capture files.
- To design a research-backed scoring system that accurately identifies threats while minimising false alerts on legitimate traffic.
- To evaluate the tool's detection performance against both normal and malicious traffic datasets and compare the efficiency of automated analysis to manual Wireshark inspection.

Objectives

1. Build an extraction module that reads packet capture files and outputs structured DNS event data in CSV format.
2. Implement a scoring engine that analyses each domain across seven detection features: Shannon entropy, TTL analysis, NXDOMAIN rate, domain name length, digit ratio, unique subdomain per parent domain, and unusual record type detection.
3. Design a multi-signal correlation system that requires at least two independent indicators to trigger before a domain can be classified as suspicious.
4. Integrate the Tranco top 100K popularity list to automatically identify well-known legitimate domains and reduce false positives.
5. Produce clear output for analysts including scored CSV reports and a single-page visual dashboard summarising the results.
6. Test the tool against both normal browsing traffic and a controlled DNS tunnelling dataset, measuring detection rate and false positive rate.
7. Compare the speed and accuracy of DNSWatch against manual Wireshark analysis of the same traffic.

2. Background and Context

Background

DNS-based attacks have grown in both frequency and sophistication in recent years. Techniques such as DNS tunnelling and domain generation algorithms, once associated with advanced threat groups, are now widely documented and accessible through open-source tools, making them available to a broader range of attackers. As more organisations move their services online, the volume of DNS traffic across networks has increased, giving attackers more opportunities to hide malicious activity within normal traffic. Despite this growing threat, many small businesses and independent analysts lack the resources and knowledge to deploy the enterprise monitoring platforms needed to detect DNS-based attacks. This gap highlights the need for lightweight, automated tools that can identify suspicious DNS patterns without requiring specialist infrastructure or expertise.

Summary of relevant published work (Literature review)

2.2.1 DNS as an attack vector

The Domain Name System (DNS) is fundamental to internet communication, translating human-readable domain names into IP addresses, and this ubiquity makes it an attractive medium for adversarial activity. Because DNS traffic is rarely blocked outright and is generally trusted at network perimeters, it can be exploited as a covert channel, a command-and-control rendezvous, or an infrastructure-hiding service [1], [2]. Salat, Davis and Khan demonstrate that DNS tunnelling and data exfiltration remain viable even in cloud environments protected by standard firewall configurations, showing that perimeter trust in DNS traffic is frequently misplaced [1]. The breadth of this attack surface is further underlined by the extensive survey of Zhauniarovich et al., which catalogues the diverse ways in which adversaries abuse DNS across the different stages of an attack life cycle [2].

Among the most documented forms of DNS abuse are DNS tunnelling, domain generation algorithms (DGAs) and fast-flux service networks. Tunnelling encodes arbitrary data within DNS queries and responses to establish a covert channel [1]. DGAs generate large numbers of candidate domain names that malware queries in search of a live command-and-control server, producing a high volume of failed lookups and randomly constructed strings [2]. Fast-flux networks rapidly rotate the IP addresses and name servers associated with a domain, defeating simple blocklists and increasing the resilience of malicious hosting [3], [4]. These techniques are formally catalogued in the MITRE ATT&CK framework as sub-techniques of dynamic resolution, specifically T1568.001 Fast Flux DNS and T1568.002 Domain Generation Algorithms [5], [6]. Industry telemetry reinforces the academic literature: Palo Alto Networks Unit 42 reports that fast flux alone is used to host phishing kits, malware distribution points and botnet infrastructure at an industrial scale [4].

This body of evidence establishes that DNS is not a passive naming utility but a live operational channel for adversaries. It is directly relevant to DNSWatch because the tool focuses exclusively on DNS rather than attempting to inspect every network protocol. By analysing DNS traffic in packet capture files offline, DNSWatch targets the single layer most consistently abused across DGA, tunnelling and fast-flux campaigns, which is why a DNS-specific analysis tool is a defensible and research-backed starting point for small-scale forensic investigation.

2.2.2 DNS threat detection methods

The literature agrees that malicious DNS activity can be identified through a combination of lexical and behavioural features, applied either to the domain string itself or to how the domain is queried and resolved [1], [2], [7]. Lexical features describe the composition of a domain name, including

length, character distribution, digit ratio and the number of distinct subdomains observed under a parent domain; behavioural features describe how the domain appears in traffic, including its time-to-live (TTL), query frequency, resolution outcome, and record types observed [2], [8].

Shannon entropy remains one of the most widely adopted lexical indicators. Wong demonstrates that algorithmically generated fully qualified domain names tend to exhibit higher character-level entropy than human-readable domains, and that a simple statistical test over entropy values can isolate DGA candidates from routine DNS traffic [7]. This observation directly motivates the inclusion of per-domain Shannon entropy as a scoring feature within DNSWatch. However, the same literature cautions that entropy alone is insufficient because legitimate content delivery network hostnames can appear artificially random [7], [8].

Building on single-feature approaches, Hajaj et al. show that a small, carefully chosen set of robust features can outperform larger feature sets when classifying malicious domains, precisely because unnecessary features introduce noise that attackers can exploit for evasion [8]. They validate that domain length; character composition and structural attributes remain reliable discriminators across datasets. Schüppen et al., in the FANCI system, focus specifically on the NXDOMAIN response class and extract structural, linguistic, and statistical features from non-existent domain traffic to detect DGA-based malware with a very low false positive rate [9]. Their approach confirms that NXDOMAIN volume and character-level abnormality, used together, provide strong DGA evidence, and it justifies the NXDOMAIN-rate feature implemented within DNSWatch.

The field also supports behavioural analysis beyond the query string. Holz et al. show that aggressively short TTL values are a reliable fingerprint of fast-flux infrastructure [3]. Professional tools such as Zeek encode many of these extraction principles into a structured DNS log, though they require network-level deployment rather than offline capture analysis [10]. These findings collectively justify the seven-feature design of DNSWatch — Shannon entropy, TTL, NXDOMAIN rate, domain length, digit ratio, unique subdomain count per parent domain and unusual record types — each of which is traceable to a specific strand of the published detection literature.

2.2.3 Challenges in DNS detection and false positives

Although multiple features can be derived from DNS traffic, the literature repeatedly cautions that each individual feature, taken in isolation, suffers from false positives [1], [2], [8], [9]. Legitimate cloud infrastructure and content delivery networks routinely display properties that resemble known attacker behaviours: very low TTLs for performance reasons, long and apparently random subdomains driven by load-balancing or analytics, and high query volumes that can mimic beaconing [1], [8]. Salat, Davis and Khan emphasise that detection is especially difficult for organisations without dedicated security expertise, because any single threshold is likely either to miss real threats or to drown analysts in benign alerts [1].

This tension was visible in the DNSWatch development cycle itself. Early experiments showed that entropy and TTL thresholds tuned against intuition produced high false positive rates on ordinary browsing traffic, flagging major content delivery and media services as suspicious. This mirrors the findings of Hajaj et al., who argue that a reduced, robust feature set is more reliable than a large, inflated one, because noisy features disproportionately affect benign traffic that happens to look unusual [8].

The prevailing response in the literature is to treat malicious DNS activity as a multi-signal pattern rather than a single-metric event. Zhauniarovich et al. conclude that robust detection combines lexical, contextual, and behavioural features before escalation rather than relying on any one family alone [2]. Schüppen et al. explicitly demonstrate that the FANCI classifier, which combines 21 features across three categories, reduces the false positive rate substantially below what any single feature achieves [9]. Holz et al. likewise use a composite metric for fast-flux detection, combining the number of distinct A records, the number of autonomous systems involved, and TTL values, rather than relying on any one of these alone [3].

This literature directly justifies one of DNSWatch's most important design decisions: the multi-signal correlation gate. Rather than flagging a domain as dangerous on the basis of a single feature crossing a threshold, DNSWatch only escalates a domain to the higher risk categories when two or more independent features fire together, applying a correlation bonus to the score when this occurs. The correlation gate is therefore not an arbitrary engineering choice but a direct response to the false positive problem documented throughout the DNS detection literature.

2.2.4 Reputation and popularity-based trust approaches

Because lexical and behavioural features alone cannot cleanly separate large legitimate services from malicious infrastructure, a parallel strand of DNS research has explored reputation and popularity data as an orthogonal context signal [11], [12]. The underlying intuition is that a domain which has been accessed by large numbers of real users over extended periods of time is unlikely to be a freshly registered attacker asset, regardless of how unusual its lexical features may appear.

The limitations of popularity rankings themselves have, however, received close attention. Scheitle et al. examine the three most widely used web top lists — Alexa, Majestic and Umbrella — and show that they exhibit substantial day-to-day instability, overlap only partially with one another, and are in some cases open to trivial manipulation [12]. Le Pochat et al. extend this analysis and demonstrate that individual popularity rankings can be altered with as little as a single HTTP request in the case of Alexa, meaning that a naive security system which trusts such a list risk treating attacker-controlled domains as popular and therefore safe [11]. In response, Le Pochat et al. introduce the Tranco list, which aggregates multiple providers, averages rankings over a configurable period, and is explicitly designed to be reproducible and resistant to manipulation [11].

This argument is directly relevant to DNSWatch, which operates in a small-footprint context where a handful of false positives on popular infrastructure can dominate analyst attention. A hand-written whitelist of trusted domains would address the symptom but would be arbitrary, difficult to justify academically, and fragile as the internet changes. Instead, DNSWatch integrates the Tranco top 100,000 list as a trust layer. Domains are still fully scored by the detection engine, but those present in the Tranco list are assigned a distinct Known-Safe category rather than being escalated to Suspicious or Dangerous.

This hybrid design mirrors the position taken in the wider literature: popularity data is treated not as a shortcut past detection, but as a stable, externally maintained source of context that complements per-domain scoring [11], [12]. It is a research-backed response to the false positive problem introduced in the previous subsection, and it is one of the most explicit links between the published literature and the final architecture of DNSWatch.

2.2.5 Gaps in existing work and relevance to this study

The review of the literature reveals a consistent pattern: individual detection features are well studied, but the tool landscape is polarised. On one side, research prototypes such as FANCI and the systems catalogued in the survey by Torabi et al. demonstrate strong analytic techniques, but they are typically tied to enterprise-scale passive DNS collection or to academic training datasets that are not available to end users [9], [13]. On the other side, professional tools such as Zeek provide excellent extraction primitives but assume a production sensor deployment and a downstream SIEM to interpret the logs [10]. Commercial offerings from vendors such as Palo Alto Networks provide mature detection but are largely opaque, exposing neither the scoring logic nor a means of reproducing results [4].

Torabi et al. make this gap explicit in their survey of implemented DNS-abuse detection systems: they observe that despite an extensive academic literature, relatively few systems are published with enough methodological detail to permit independent reproduction, and fewer still are designed for lightweight forensic use by analysts without enterprise infrastructure [13]. Zhauniarovich et al. reach a similar conclusion, arguing that the distance between published detection algorithms and field-ready tools remains one of the persistent weaknesses of the area [2].

DNSWatch is positioned precisely in this gap. It takes features whose foundations are documented in peer-reviewed work — entropy [7], TTL-based fast-flux indicators [3], NXDOMAIN patterns [9], robust lexical features [8] and multi-signal correlation [2], [9] — and implements them in a small offline Python tool that consumes packet capture files and produces transparent scoring. The use of Tranco [11] as a trust layer addresses the practical false positive problem that would otherwise undermine such a tool. In this sense, DNSWatch is not an arbitrary implementation, but a direct synthesis of the themes reviewed above, aimed at the underserved intersection of research-backed detection, methodological transparency, and lightweight offline use.

Competitor Analysis/ Related work

2.3 Competitor Analysis / Related Work

Several existing tools and platforms can be used to inspect or secure DNS traffic, but they differ significantly in cost, complexity, transparency, and intended use. The most relevant related tools for this project are Wireshark, Zeek, Graylog, and Cisco Umbrella. Each offers useful capabilities, but none matches the exact combination of accessibility, lightweight deployment, and transparent DNS-focused scoring provided by DNSWatch.

2.3.1 Wireshark

Wireshark is a widely used open-source network protocol analyser that allows users to capture and inspect network packets in detail. It is particularly valuable for manual DNS inspection because analysts can filter traffic, inspect query and response fields, and observe protocol behaviour directly at packet level. This makes Wireshark highly effective for low-level investigation and validation work. In the context of this project, Wireshark was also important as a supporting tool because it underpins the use of TShark, the command-line packet parser used in DNSWatch's extraction stage. Wireshark describes itself as a powerful open-source network protocol analyser capable of deep inspection across hundreds of protocols.

However, Wireshark's main limitation is that it is largely a manual analysis tool. Although it provides extensive visibility, it does not automatically score domains, classify them by severity, or summarise DNS findings in the way required for this project. An analyst must manually inspect filters, examine packet contents, and interpret suspicious behaviour for themselves. This makes Wireshark useful for detailed packet-level analysis, but less suitable when a user needs a fast and simple way to assess the severity of DNS traffic from a packet capture file. DNSWatch addresses this limitation by automating extraction, feature scoring, and visual reporting, while still using Wireshark-compatible technology through TShark.

2.3.2 Zeek

Zeek is an open-source network security monitoring platform that generates structured logs from observed network traffic. Its DNS logging capabilities are especially relevant because `dns.log` is presented in Zeek's documentation as one of its most important data sources for security and network administration. This makes Zeek an important related work example because, like DNSWatch, it treats DNS traffic as a valuable source of security evidence rather than only as background network noise.

The key strength of Zeek is that it produces rich, structured telemetry that can support detection engineering and forensic analysis across many protocols, not just DNS. However, this broad capability also introduces more setup complexity and a steeper learning curve. Zeek is not a lightweight single-purpose DNS scoring tool; it is a full network security monitoring framework intended for analysts who can deploy and interpret its logs at scale. In addition, while Zeek produces detailed DNS evidence, it does not openly present a transparent DNS scoring model equivalent to DNSWatch's seven-feature classification logic. For this reason, Zeek is better

understood as a powerful open-source monitoring framework related to the project, rather than a direct substitute for the final artefact. DNSWatch occupies a narrower but more accessible role by focusing on offline DNS analysis from PCAP files and providing immediate scored outputs for the user.

2.3.3 Graylog

Graylog is positioned as a SIEM and log management platform that centralises and monitors machine data for security and operations use cases. Its official materials describe it as a SIEM, log management, and security analytics solution intended to improve visibility, reduce alert noise, and accelerate investigations. This makes Graylog relevant because it represents the kind of broader enterprise monitoring approach that many organisations may use to analyse security events, including DNS-related data if the appropriate logs are ingested and processed.

Despite these strengths, Graylog differs from DNSWatch in both scope and accessibility. Graylog is designed as a centralised platform that depends on log collection, integration, correlation, and ongoing operational management. While this is powerful in enterprise settings, it introduces infrastructure and knowledge requirements that may be disproportionate for smaller businesses or individual analysts who only need to inspect a capture file and understand whether suspicious DNS behaviour is present. Graylog also operates at a broader SIEM level, whereas DNSWatch is intentionally focused on one specific task: analysing DNS behaviour from pre-captured traffic in a clear and explainable way. This difference is important because it reflects the problem statement of the project itself: enterprise platforms are powerful, but they may be too heavy, costly, or complex for users who need a lightweight alternative.

2.3.4 Cisco Umbrella

Cisco Umbrella is one of the most relevant commercial comparisons because it is explicitly positioned as a DNS-layer security product. Cisco describes Umbrella as using DNS-layer security to block threats before connections are fully established, and also states that its protective DNS service uses Cisco Talos threat intelligence to stop attacks before they reach the network. This makes Umbrella highly relevant to DNSWatch because both solutions focus on the security value of DNS traffic.

However, Cisco Umbrella is fundamentally different from DNSWatch in purpose and deployment model. Umbrella is a cloud-delivered commercial security platform aimed at live protection and enforcement across networks, users, and devices. It is designed to block access to malicious destinations in real time, whereas DNSWatch analyses pre-captured packet data offline. Umbrella is therefore closer to an enterprise defensive service than to a lightweight research artefact. A further challenge from an academic perspective is that commercial tools such as Umbrella do not openly publish the detailed internal scoring logic used to classify suspicious DNS traffic. This makes direct methodological comparison difficult. DNSWatch differs by being transparent and explainable: the scoring features, thresholds, and classification outputs are visible to the user and can be directly linked back to the literature and the implementation.

2.3.5 Positioning DNSWatch against existing tools

When compared with these related tools, DNSWatch occupies a distinct middle ground. Wireshark offers excellent packet-level visibility but relies heavily on manual interpretation. Zeek provides rich structured logs and broad monitoring capability, but requires greater expertise and setup effort. Graylog offers enterprise-scale SIEM and analytics functionality, but its broader infrastructure demands may make it unsuitable for the lightweight use case targeted in this project. Cisco

Umbrella provides live DNS-layer protection, but as a commercial security platform it does not offer the same transparency or simplicity of deployment as a small standalone artefact.

DNSWatch was developed to address a narrower but important need: enabling a user to take an existing packet capture, extract DNS activity, apply a research-backed scoring model, and receive a clear classification of suspicious behaviour without deploying an enterprise platform. Its main contribution is therefore not that it replaces large-scale monitoring tools, but that it makes DNS threat analysis more accessible, explainable, and portable. This is especially relevant for smaller organisations, students, or analysts who need an interpretable offline DNS analysis tool rather than a full SIEM ecosystem. In this sense, DNSWatch complements existing tools rather than attempting to compete with them directly across all security functions.

3.Tools and Methods

Rationale for the technical tools selected

DNSWatch was developed using Python 3.12, chosen for its readability and the availability of mature libraries for data processing and analysis. Python was also recommended by the project supervisor as a suitable language for this type of network security tooling. Alternative languages such as Java or C++ were considered but rejected due to their steeper learning curves and longer development times for a project of this scope.

The extraction layer relies on tshark, the command-line interface of Wireshark, to parse DNS packets from PCAP and PCAPNG capture files. Tshark was chosen because it uses the same protocol dissection engine as Wireshark, this ensures that DNS fields are parsed accurately and consistently. An earlier version of the extraction module used the Scapy library, a Python-based packet manipulation tool. However, during testing Scapy was found to be unreliable when reading PCAPNG files and produced inconsistent results with certain capture formats. Tshark resolved these issues and provided better alignment with the project's academic methodology, as Wireshark is an industry-standard tool widely referenced in network security literature.

The detection layer uses pandas for data processing and grouping operations. Pandas was selected because it provides efficient tools for reading CSV files, aggregating data by domain name, and performing the grouped calculations required for per-domain scoring. The tldextract

library was used to reliably extract parent domains from full query names, which was necessary for the Tranco popularity lookup.

Visualisations were generated using matplotlib, a standard Python plotting library. Matplotlib was chosen because it supports the creation of multi-panel figures and can output high-resolution PNG files without requiring a graphical user interface, which is important for a command-line tool. The Tranco top 100K domain popularity list was integrated as an external data source to provide a research-backed trust layer, as discussed in the literature review.

Rationale for the research methods used

This project used a design science research approach; a methodology suited to projects where the primary goal is to create and evaluate a practical artefact rather than to test a hypothesis through observation alone. Design science research involves identifying a real-world problem, designing a solution to address it, building the solution, and then evaluating its effectiveness through structured testing. If the evaluation reveals weaknesses, the design is rebuilt and tested again. This iterative cycle continues until the artefact satisfactorily addresses the original problem.

Design science is widely used in computer science and information systems research because it focuses on producing useful, working outputs rather than purely theoretical findings. It is particularly well suited to software development projects where requirements may evolve as the developer gains a better understanding of the problem through building and testing.

This methodology was found appropriate for DNSWatch because the project's core challenge was not simply understanding DNS threats in theory; but instead building a tool that could detect them effectively in practice. The first version of the detection engine used three scoring features and produced a high rate of false positives on normal browsing traffic. Testing against real captured data revealed that legitimate CDN infrastructure was being incorrectly flagged. This led to a redesign where seven detection features, a multi-signal correlation requirement, and a Tranco-based trust layer was implemented. The revised version was then tested again against both normal and malicious traffic, producing significantly improved results. This cycle of design, implementation, evaluation, and refinement is characteristic of design science and directly shaped the final artefact.

Project management and Product lifecycle

The project was managed using an informal iterative approach rather than a formal project management framework. Development was organised into focused working sessions, each targeting a specific area of progress such as building the extraction layer, implementing detection features, or refining thresholds based on test results. While a formal tool such as Trello or GitHub Projects was not used to track tasks, the project maintained a clear sense of progression through regular supervisor meetings and ongoing documentation of design decisions.

The development lifecycle followed an iterative model consistent with the design science methodology described above. Rather than planning the entire system in advance and building it in one pass, each component was developed, tested against real traffic data, and refined before moving to the next stage. For example, the detection engine went through two major iterations: the first version used three scoring features and was tested on a normal browsing capture, revealing a high false positive rate. The second version expanded to seven features with a multi-signal correlation gate and was tested against both normal and malicious traffic, producing significantly improved results. This iterative approach allowed the project to respond to practical findings rather than being locked into an early design that had not yet been validated.

Supervision meetings with the project supervisor took place regularly throughout the development period and were used to review progress, discuss design decisions, and incorporate feedback. Key decisions such as the redesign of the scoring system, the adoption of the Tranco popularity list, and the consolidation of the dashboard output were all influenced by supervisor feedback. These meetings provided structure and accountability that complemented the informal task management approach.

Legal, ethical and Professional issues

The most significant consideration in this project was the capture and analysis of network traffic data. The normal browsing traffic used for testing was captured on the researcher's own home network, ensuring no third-party traffic was intercepted or analysed. The malicious traffic dataset was a publicly available dnscat2 capture downloaded from an open research source containing no personally identifiable information.

The project was conducted in compliance with the Computer Misuse Act 1990, as all analysis was performed on data the researcher either generated themselves or obtained from a legitimate public source. The project also aligns with the principles of UK GDPR in that no personal data belonging to third parties was collected or processed.

DNSWatch operates as a passive forensic analysis tool that does not intercept live traffic, modify network behaviour, or interact with external systems. This positions it as a defensive tool consistent with professional standards in cybersecurity. An ethics form was completed and approved prior to the start of the project and is included in the appendices.

Risk register

Below is a table of a risk register:

Table 1 Risk Register

Risk	Likelihood	Impact	Mitigation
Loss of project files due to hardware failure	Medium	High	All project files stored in OneDrive with automatic cloud synchronisation across two devices (Windows PC and MacBook Air)
Tshark parsing errors producing incorrect DNS data	Medium	High	Extraction output was manually validated against Wireshark's own GUI to confirm field accuracy before any detection logic was applied
High false positive rate reducing the usefulness of detection output	High	High	Identified early through testing on real traffic. Addressed through iterative threshold tuning, multi-signal correlation

			requirement, and Tranco-based trust layer
Dependency issues preventing the tool from running on different operating systems	Medium	Medium	Cross-platform compatibility was built into main.py using automatic OS detection for the tshark path. Tested on both Windows and Mac
Scope creep delaying completion of the artefact	Medium	High	Scope was managed through regular supervisor meetings. Features such as MISP integration and DNS over HTTPS analysis were identified as valuable but deferred to future work rather than risking incomplete implementation
Inability to test against real malicious traffic	Medium	High	A publicly available dnscat2 DNS tunnelling dataset was sourced to provide a controlled malicious traffic sample for evaluation

4. Requirements/Specifications

Persona:

Below is a persona of a IT administrator for DNSWatch:

Persona: Sarah, Small Business IT Administrator

Age: 32

Role: IT Administrator at a small marketing agency with 15 employees

Technical level: Comfortable with basic networking but no dedicated team or specialist training

Problem: Occasionally captures network traffic when something feels wrong but has no way to quickly identify whether DNS activity is suspicious without spending hours in Wireshark

Goal: A fast, simple way to run a packet capture through a tool and get a clear answer about whether anything looks malicious without having to deploy enterprise level software or read through hundreds of raw packets manually

Why DNSWatch fits: Free, offline, no installation of complex infrastructure, produces a clear scored output and dashboard she can interpret without specialist knowledge.

Functional requirements:

Below is a table of the functional requirements of DNSWatch:

Table 2 Functional Requirements

ID	Requirement	Justification
FR1	The system shall read PCAP and PCAPNG packet capture files as command line input	Enables analysis of pre-captured traffic without requiring live network access
FR2	The system shall extract structured DNS event data and output it as a CSV file	Produces a reusable, readable dataset that can be inspected independently of the detection layer
FR3	The system shall analyse each unique domain across seven detection features: Shannon entropy, TTL, NXDOMAIN rate, domain length, digit ratio, unique subdomain count and unusual record types	Ensures detection is based on multiple measurable indicators rather than a single threshold
FR4	The system shall require at least two independent features to fire before classifying a domain as Suspicious or Dangerous	Prevents false positive caused by legitimate services triggering a single feature for operational reasons
FR5	The system shall cross-reference scored domains against the Tranco top 100K popularity list and assign a Known-Safe category to recognised domains	Allows an analyst to focus immediately on the highest priority findings
FR6	The system shall produce an alerts CSV containing only Suspicious and Dangerous domains sorted by score	Allows an analyst to focus immediately on the highest priority findings
FR7	The system shall produce a four-panel visual dashboard	Provides an accessible summary of findings without

	as a PNG file summarising the analysis results	requiring the analyst to interpret raw CSV data
--	---	--

Non-Functional requirements:

Below is a table of non-functional requirements for DNSWatch

Table 3 Non-Functional requirements

ID	Requirement	Justification
NFR1	The system shall operate entirely offline without requiring internet access during analysis	Ensures the tool can be used in restricted or air-gapped environments
NFR2	The system shall run on both Windows and macOS without modification	Ensures portability across the platforms most commonly used by small business analysts
NFR3	The system shall produce scoring outputs that are fully explainable, showing which features triggered for every domain	Ensures analysts can verify and understand every classification without treating the tool as a black box
NFR4	The system shall require no enterprise infrastructure, licensing costs or specialist deployment knowledge	Ensures accessibility for small businesses and individual analysts as identified in the problem statement
NFR5	The system shall process a standard packet capture file and produce all outputs within a timeframe suitable for interactive use	Ensures the tool provides a meaningful efficiency improvement over manual Wireshark analysis

Use Case Diagram:

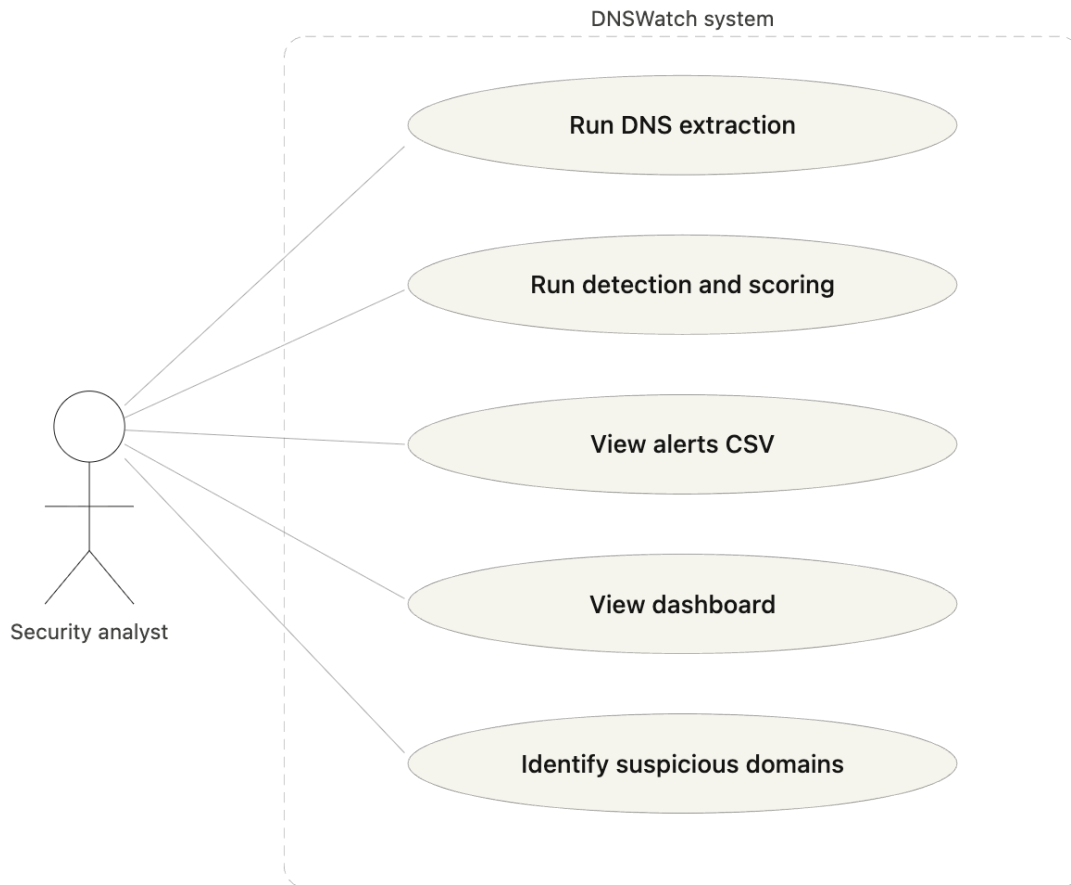


Figure 1 Use Case Diagram

5. Solution design

Workflow diagram:

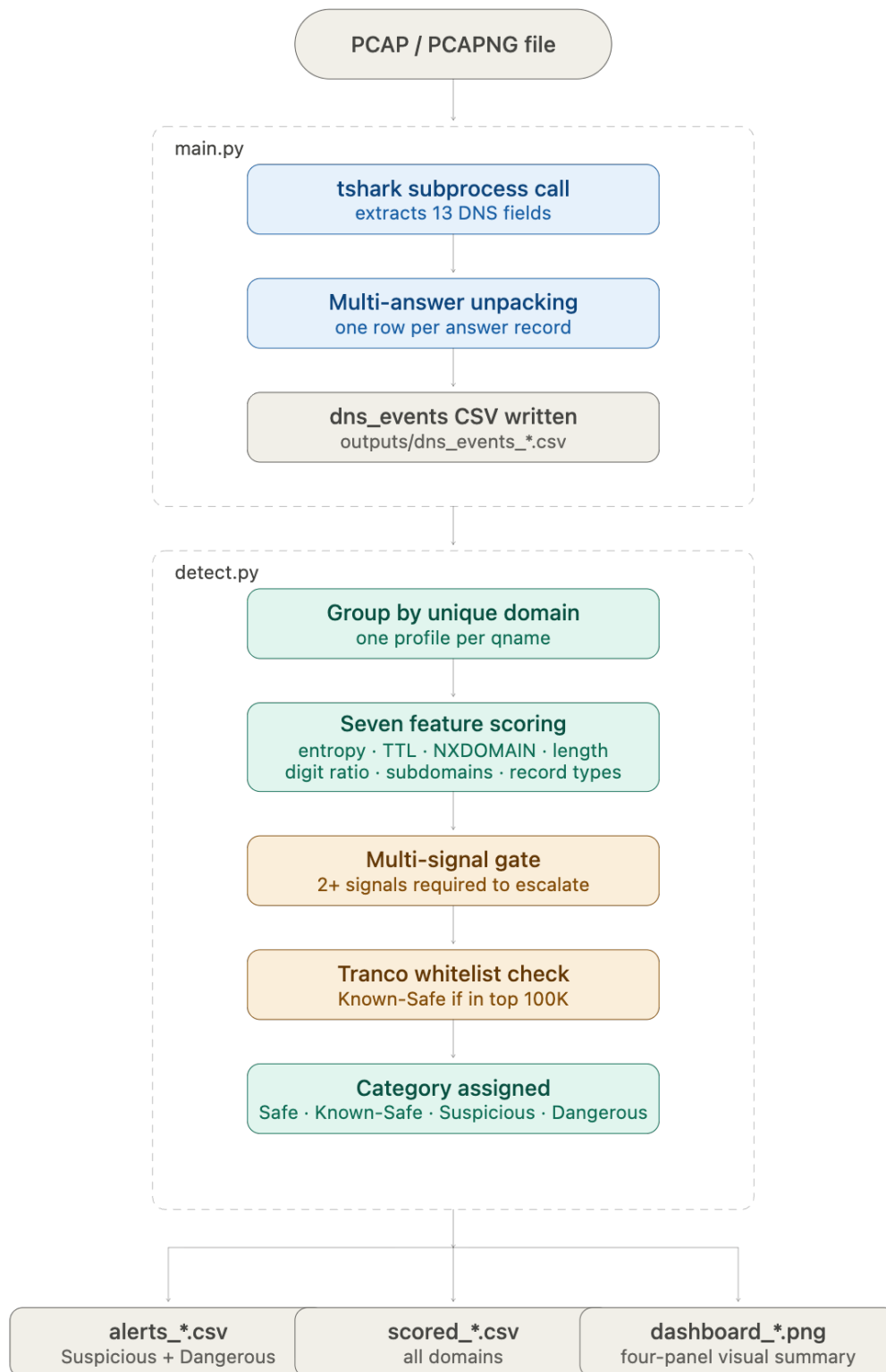


Figure 2 Workflow Diagram

6. Solution implementation

DNSWatch was developed iteratively across two main phases of work, each phase targeting a distinct layer of the system. The first phase focused on building a reliable DNS extraction module, whereas the second phase focused on developing and refining the detection and scoring engine. This work structure reflects the design science approach that was described in the Tools and Methods section; each phase ended with testing against real traffic and the results of each test directly shaped the next phase of development.

Sprint 1 – DNS Extraction Layer (main.py):

The first task at hand was to build a module that was capable of reading packet capture files and producing it into structured DNS data that could be analysed programmatically. The project folder and module structure was established at this stage, with `src/`, `/data`, and `outputs/` directories being created to separate concerns from the outset and tidiness.

The initial approach taken was to use Scapy, which is a Python-based packet manipulation library, to parse DNS fields directly. However, during early testing Scapy came out to be inconsistent with its results when reading PCAPNG files and failed to handle certain multi-answer DNS responses reliably. Rather than attempting to work around these limitations of Scapy, the decision was made to replace Scapy entirely with `tshark`, which is a command-line interface of Wireshark itself, making it a more academically defensible choice and it ensures that the DNS field extraction was consistent with industry-standard tooling.

The `tshark` subprocess call at the core of `main.py` extracts thirteen DNS fields per packet in tab separated format:

```
def run_tshark(pcap_file):
    """
    Executes Tshark to parse DNS packets from the given PCAP/PCAPNG file.
    Returns a subprocess.Popen object if successful, None otherwise.
    """
    cmd = [
        TSHARK_PATH,
        "-r", pcap_file,
        "-Y", "dns",
        "-T", "fields",
        "-e", "frame.time_epoch",
        "-e", "ip.src",
        "-e", "ip.dst",
        "-e", "dns.flags.response",
        "-e", "dns.qry.name",
        "-e", "dns.qry.type",
        "-e", "dns.flags.rcode",
        "-e", "dns.resp.name",
        "-e", "dns.resp.type",
        "-e", "dns.resp.ttl",
        "-e", "dns.a",
        "-e", "dns.aaaa",
        "-e", "dns.cname",
        "-E", "header=n",
        "-E", "separator=/t",
        "-E", "quote=n",
        "-E", "occurrence=a",
        "-E", "aggregator=",
    ]
```

Figure 3 DNS packet parsing

The most significant technical challenge faced in this phase was the handling of DNS responses that contain multiple answer records, a single DNS response can return several different IP addresses, CNAME chains or a mixture of record types simultaneously. Tshark outputs these as a comma-separated values within a single field, which required a custom unpacking routine to split each answer into its own row in the output CSV. A custom unpacking routine was written to help ensure that even when the answer fields contain different numbers of records, every answer is written correctly without data being dropped or misaligned.

By the end of Sprint 1, main.py was successfully extracting 504 DNS event rows from a thirty-second normal browsing capture, validated against Wireshark's own output to confirm field accuracy.

Sprint 2 – Detection and Scoring Engine (detect.py):

With now a reliable extraction layer in place, the second phase really focused on building the detection logic. The first version of detect.py applied scoring on a per-row basis where every row in the CSV was scored individually, and any row crossing a threshold produced an alert.

The first version of the scoring engine contained three features initially: Shannon entropy, minimum TTL and NXDOMAIN Rate. Each feature had a single threshold, for example entropy above 3.8 scored 35 points, TTL below 60 seconds scored 25 points, and an NXDOMAIN rate above 0.5 scored 35 points. These thresholds were chosen based on initial research but hadn't yet been

validated against real traffic, when it came to testing against the normal browsing capture it quickly revealed that the thresholds were too aggressive as legitimate CDN Infrastructures were triggering the entropy and TTL signals for valid reasons such as operational reasons. As a result, the entropy threshold was then raised from 3.8 to 4.0 and the TTL threshold was also lowered from 60 seconds to 30 seconds based on what the real traffic revealed. However, even after these adjustments were made the false positives persisted, as the first test run had produced 92 alerts including 16 classified as “Dangerous”, which every one of them being a legitimate service including Google, BBC and Steam CDN infrastructure. The root of this problem was that the scoring was working per-row rather than per-domain which led to the redesign from per-row to per-domain.

The core of this redesign involved moving from a per-row scoring to per-domain scoring, rather than scoring each row independently, detect.py v2 firstly groups all rows by the unique domain name using pandas, as a result building one consolidated profile per domain:

```
# --- Step 2: Group by unique qname -----
def get_mode(series):
    m = series.mode()
    return m.iloc[0] if not m.empty else None

grouped = df.groupby("qname", as_index=False).agg(
    first_seen_ts=("timestamp", "min"),
    query_count=("timestamp", "count"),
    min_ttl=("ttl", "min"),
    nxdomain_count=("rcode", lambda s: int((s == 3).sum())), # type: ignore[union-attr]
    src_ip=("src_ip", get_mode),
)
```

Figure 4 Group by Per Unique Domain

This means that instead of scoring 504 individual different rows, the tool now builds one profile per unique domain, therefore capturing its minimum TTL, total query count, NXDOMAIN rate across all observations and lastly most frequent source IP.

This single change itself transformed the outputs from 92 duplicate alerts to 43 unique domain profiles, now giving one verdict per domain rather than one alert per packet. Detection features were also expanded from three to seven: Shannon entropy, TTL analysis, NXDOMAIN rate, domain name length, digit ratio, unique subdomain count per parent domain and unusual record type detection.

DNSWatch v2 Scoring Specification

Below is a table of how DNSWatch scores the domains it extracts:

Table 4 DNSWatch v2 Scoring Specification

Feature	Condition	Score
Shannon Entropy	Above 4.0	30 points
Shannon Entropy	Above 3.5	20 points
TTL	Below 30 seconds	20 points
TTL	Below 300 seconds	10 points
NXDOMAIN rate	Above 0.5	30 points
NXDOMAIN rate	Above 0.3	20 points
Domain length	Above 75 characters	25 points
Domain length	Above 50 characters	15 points
Digit ratio	Above 0.4	20 points
Digit ratio	Above 0.3	10 points
Unusual record types	NULL, TXT or ANY present	15 points
Multi-signal bonus	3+ features triggered	+10 points

(Maximum score capped at 100. Categories: 0-29 = Safe, 30-59 = Suspicious, 60-100 = Dangerous. Domains present in the Tranco top 100K list are assigned Known-Safe regardless of the score)

A multi-signal correlation was introduced as a safeguard against false positives, now a domain whose score comes entirely from a single feature is capped at 29 points, which is below the Suspicious threshold, regardless of how high that single signal scores. Only when two or more independent features fire simultaneously is when the domain becomes eligible for an escalated score, with a bonus also applied when three or more signals firing together. This directly addressed the CDN false positive I encountered where legitimate services may trigger one signal for a valid operational reason but however, they would be unlikely to trigger several simultaneously.

The Tranco top 100,000 domain popularity list was also integrated as a final trust layer, domains which are present in the Tranco list are assigned with a Known-Safe category rather than being escalated regardless of their score they accumulated. This was chosen over a manually written whitelist because the Tranco methodology is academically grounded and externally maintained.

After all these changes, retesting had begun against the same normal browsing captured and produced zero false positives, then testing the same changes against a public dnscat2 DNS tunnelling dataset of approximately 6,900 domains produced a detection rate of 99.87%

Dashboard Output:

During the final clean up phase, the visualisation output was finalised from multiple individual chart files, turning it into a single four-panel dashboard PNG. The four panels contained: Domain Category Distribution, Detection Signal Frequency, Features Triggered Per Domain and Top 10 Most Queried Domains. This gives an analyst an immediate summary of the capture without needing to go further in depth in the CSV files. This decision also significantly reduced the output clutter and made the tool's results more accessible for a non-specialist user.

The dashboard below shows the results of running DNSWatch against the normal browsing capture, with 43 domains analysed and zero classified as Suspicious or Dangerous.

DNSWatch Analysis — sample | Domains: 43 | Safe: 28 | Known-Safe: 15 | Suspicious: 0 | Dangerous: 0 | Generated: 2026-03-17

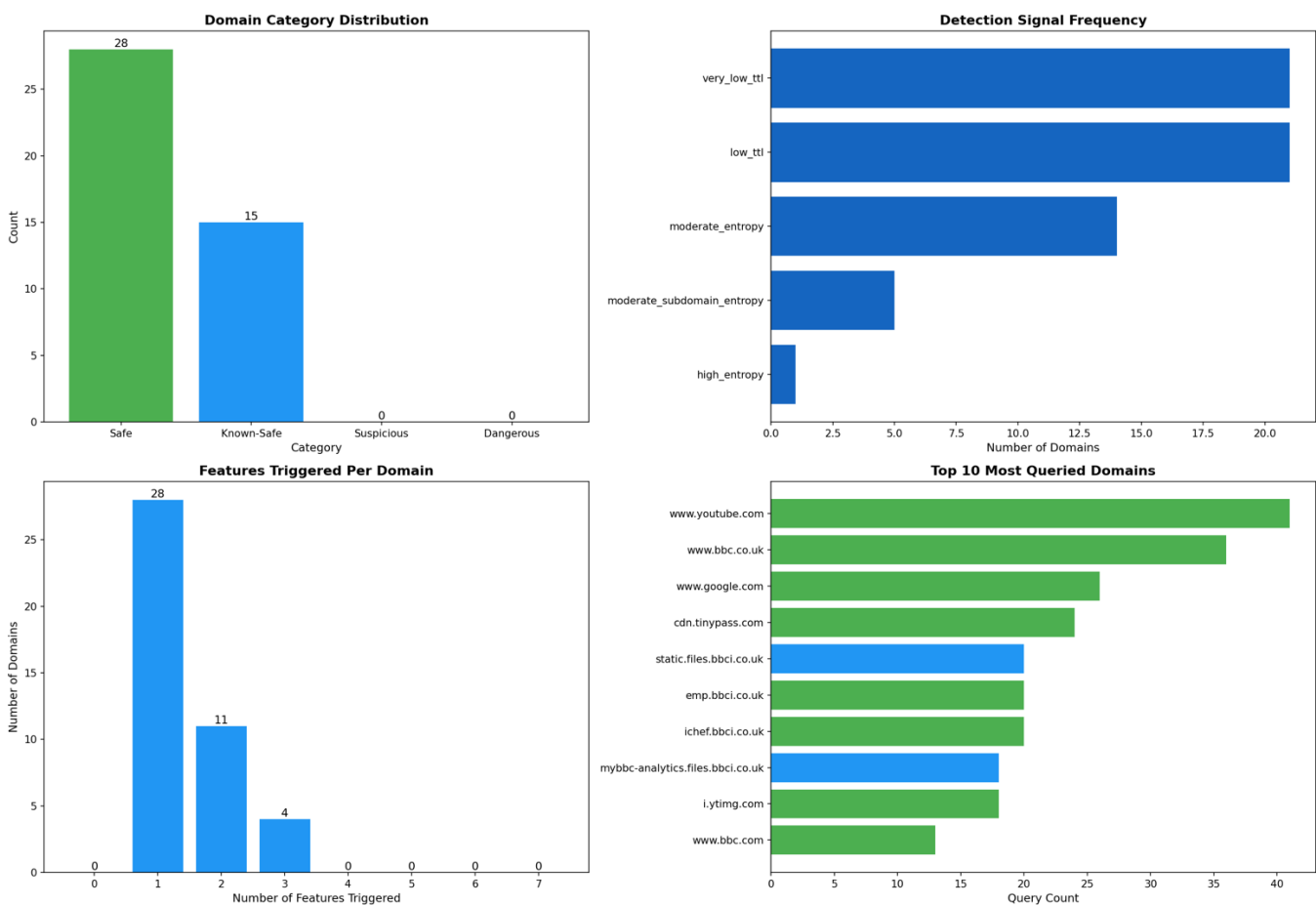


Figure 5 Normal traffic dashboard

The dashboard below shows the results of running DNSWatch against the dnscat2 DNS tunnelling dataset, with 6,904 domains analysed and 6,895 correctly classified as Dangerous.

DNSWatch Analysis — dnscat2_dns_tunneling_1hr | Domains: 6904 | Safe: 3 | Known-Safe: 1 | Suspicious: 5 | Dangerous: 6895 | Generated: 2026-03-17

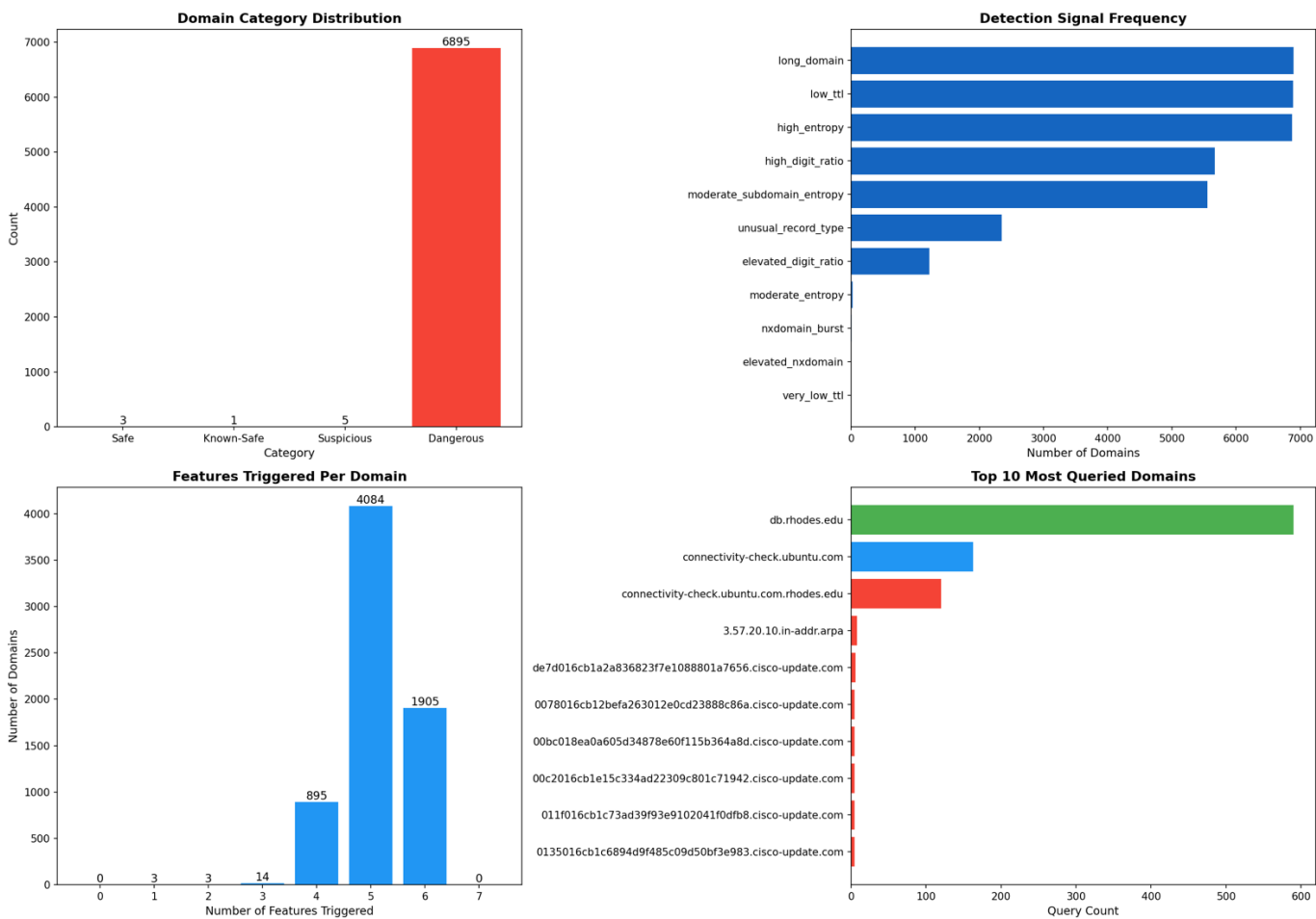


Figure 6 Dashboard for Malicious traffic

7. Solution testing/ evaluation

The evaluation of DNSWatch was conducted using two very distinct datasets with each of them being designed to measure a different aspect of the tool's capabilities and performance. The first dataset tested the false positive rate against normal traffic, whereas the second was testing the detection rate against a known malicious DNS tunnelling traffic. Together these two tests address the core challenge identified in the literature review, that a DNS detection tool must be both sensitive enough to catch real threats and also specific enough to avoid overwhelming any analysts with false alerts.

Test 1 – Normal browsing capture (false positive rate):

The first dataset was just a packet capture recorded on the researcher's own home network, it consisted of a thirty second normal browsing session. The session included visits to websites such as Google, BBC, YouTube, Twitter, and Steam which is a representative sample size of everyday web activity for majority of the population that would be expected to produce a variety of legitimate DNS queries across different types of infrastructures such as search engines, media platforms, gaming services etc.

main.py extracted 504 DNS event rows from this 30 second capture, detect.py v2 analysed these rows and identified 43 unique domains. After applying the seven-feature scoring model, the multi-signal correlation gate and the Tranco trust layer there was zero domains in this capture which were classified as Suspicious or Dangerous, representing a false positive rate of 0%.

This result was a significant improvement over the detect.py v1, which itself produced 92 alerts including 16 of them being classified as Dangerous on the same traffic while every one of the 16 was a legitimate service. The improvement demonstrates that the combination of per-domain scoring, refined thresholds, multi-signal correlations and the Tranco whitelist successfully eliminated the false positive problem that was encountered during early phases of development.

Test 2 – dnscat2 DNS Tunnelling Capture (Detection Rate):

The second dataset was a publicly available one-hour DNS tunnelling capture that was produced using the dnscat2 tool, sourced from Active Countermeasures [14]. The dnscat2 tool, developed by

Ron Bowes [15], Is a widely used open-source DNS tunnelling tool that encodes arbitrary data within DNS queries and responses to establish a covert communication channel. The capture was generated in a controlled lab environment, and it contains no personally identifiable information.

detect.py v2 analysed approximately 6,900 unique domains from the unique dataset of dnscat2 DNS tunnelling capture; of these 6,900, 99.87% were correctly classified as Dangerous. These results demonstrate that the scoring model is highly effective at detecting DNS tunnelling activity, which was the primary attack type the tool itself was designed to identify.

This strong detection rate is explained due to the nature of dnscat2 traffic, DNS tunnelling produces domains with extremely high Shannon entropy due to encoded payloads, unusually high unique subdomain counts under a single parent domain and query patterns inconsistent with normal browsing behaviour. Due to these characteristics, multiple features fire simultaneously, triggering the multi-signal correlation gates and producing high scores consistently across tunnelling domains.

DNSWatch evaluation results against standard detection metrics

Below is a table of evaluation results against standard detection metrics:

Table 5 DNSWatch evaluation results

Metric	Result
Detection rate (true positive rate)	99.87%
False positive rate	0% (normal browsing capture)
Processing approach	Automated offline analysis vs manual Wireshark inspection

The processing comparison to a manual Wireshark analysis is qualitative rather than timed. Manually identifying and inspecting 43 different domains across 504 packets in Wireshark would require an analyst to apply filters, read the different individual records and making judgements without any scoring system to guide the prioritisation. DNSWatch automates this whole process and produces a score, categorised output in seconds and representing a meaningful reduction in manual effort even without a precise timing measurement.

Strengths:

The tool demonstrated strong performances across both test datasets, with a false positive rate of 0% on normal browsing traffic means an analyst using DNSWatch on everyday network captures wouldn't be presented with misleading alerts. The detection rate of 99.87% on DNS tunnelling

traffic demonstrates that the improved scoring model is extremely effective at identifying the type of activity it was designed to detect in the first place. The transparency of the scoring system is a plus, as every classification includes a reasoning showing exactly which features fired, leading an analyst to understand and verify any alerts without having to treat the tool as a black box. The tool also requires no enterprise infrastructure, ongoing configuration and operates entirely offline from a single standard PCAP file.

Weaknesses / Limitations:

There are several limitations that must be acknowledged of DNSWatch, one being it analyses pre-captured traffic only and doesn't operate as a live monitoring system, meaning it can't provide real time alerts. The evaluation was conducted against a single attack type being DNS tunnelling, while the literature supports the DNSWatch's feature design for detecting DGA and fast-flux activities, they were not formally tested against dedicated datasets which therefore limits the scope of the evaluation. The normal browsing capture that was used for false positive testing was only thirty seconds long and was generated on a single home network, which may not represent the full variety of legitimate traffic an analyst would have to encounter in a real-world environment. The Tranco top 100K list will require periodic updating to make sure it remains as accurate as possible as domain popularities do change.

8. Discussion

The results produced by DNSWatch across both of the dataset test demonstrate that a lightweight, rule-based DNS analysis tool that is built on research backed features can achieve strong performance without an enterprise infrastructure. It also reveals that the core design decisions that have been made throughout development were key and well founded, as they have led to a false positive rate of 0% and detection rate of 99.87%.

The most significant finding is the move from per-row to per-domain scoring combining with the multi-signal correlation gate as it was the single most impactful design decision. Threshold adjustments alone proved to be insufficient during developing, the key change was requiring multiple independent signals to fire before escalating the severity, meaning legitimate services that trigger one feature for valid operational reasons could no longer be misclassified. This therefore directly supports the conclusions of Zhauniarovich et al. and Schüppen et al., both of whom argue that robust detection requires combining multiple feature families rather than relying on any single indicator.

The 99.87% detection rate on dnscat2 dataset confirms that DNS tunnelling produces a highly detectable signal pattern, the results validate the argument made by Hajaj et al. that a small carefully chosen feature set can be just as effective as a larger one while also remaining simpler to explain. DNSWatch achieves a comparable false positive rate to FANCI's 21-feature classifier while only using seven features compare to the twenty-one.

The project provides practical evidence for two different arguments from the literature: firstly, being that DNS tunnelling is detectable through behavioural features alone without threat intelligence feeds, and secondly that single feature detection is genuinely inadequate for real world scenarios and traffic. The results from v1 demonstrated this, confirming that the literature predicts in theory. The limitations of the evaluation being a single attack type and two small datasets point directly toward the further work in the future that is discussed in the conclusion.

9. Conclusion

Summary

This project set out to address the gap between enterprise-grade DNS monitoring platforms and manual packet analysis tools by developing DNSWatch; a lightweight Python-based DNS traffic analysis tool. The tool was built across two development phases, an extraction layer using tshark to parse DNS fields from packet capture files, and also a detection layer implementing a seven-feature scoring system with a multi-signal correlation gate and tranco based trust layer implemented. It was evaluated against two datasets, one being a normal browsing capture and the other a public dnscat2 DNS tunnelling dataset, producing a false positive rate of 0% and a detection rate of 99.87%. These results demonstrate that a research-backed, transparent, and lightweight DNS analysis tool can achieve a strong performance in detection without requiring any complex enterprise infrastructure or specialist knowledge for deployment.

Recommendations for further work

Several improvements for DNSWatch were identified during the development and evaluation phases that couldn't be implemented within the timeframe and scope of this project. The most immediate recommendation would be to expand the attack types beyond DNS tunnelling, specifically towards DGA-based malware and fast-flux infrastructures, as these were not specifically tested against dedicated datasets despite the detection features being designed with them in mind. A broader and more diverse testing dataset would strengthen the confidence of the tool's detection capabilities.

A second recommendation would be to integrate an optional MISP threat intelligence feed, which was intended early in development as a valuable enrichment layer but deferred away from it to avoid scope creep. This addition would allow DNSWatch to cross-reference scored domains against known malicious indicators from the community threat intelligence sources which would help a reputation-based signal to complement the existing scoring system.

Finally, a live capture mode would completely transform DNSWatch from a forensic analysis tool into a lightweight real time monitoring solution, addressing the most significant limitation that has been identified in the evaluation.

Reflection

Looking back on this project, the most significant challenge that I had faced was not technical but personal. Early in the academic year I had my mind set on the original project idea of: SafetyNET, which was abandoned after it became clear to me that the direction, I was heading into with it was not one I found motivating or satisfying. Changing projects at a late stage was a difficult decision, but it was the right one. DNSWatch emerged from a genuine interest in understanding how DNS traffic could be used as a security signal, and that motivation carried the project through to completion.

The switch also came with real consequences, the time I had spent on the previous idea couldn't be recovered which meant that the DNSWatch development timeline was compressed from the beginning. In hindsight, if I've recognised earlier that the project needs to be something that the developer genuinely cares about I would have completely avoided this pressure, so If I were to start again I would spend more time at the proposal stage making sure the idea was something I could sustain my interest in across the academic year rather than committing to the first viable idea that came up in my head.

From a technical viewpoint, this project taught me a great deal that I did not know at the start. I had a general foundational understanding of DNS before beginning, but building DNSWatch gave me a much deeper appreciation of how DNS traffic behaves in practice, how attackers exploit it and how measurable features can be derived from query patterns. Researching tools like Graylog, Splunk and Cisco Umbrella also gave me a clearer picture of why enterprise monitoring platforms, despite their power are inaccessible to smaller organisation and that gap to me was quite interesting and worth addressing.

What I am most proud of however is delivering a complete, working and well thought out tool after switching projects so late. DNSWatch achieves a 0% false positive rate and a 99.87% detection rate, it produces a clean four panel dashboard and is grounded in peer-reviewed literature throughout. This outcome, given the circumstances I had to go through, represents the most demanding and rewarding piece of work I have completed during my tenure in this degree.

10. References

- [1] L. Salat, M. Davis, and N. Khan, "DNS Tunnelling, Exfiltration and Detection over Cloud Environments," *Sensors*, vol. 23, no. 5, Art. no. 2760, Mar. 2023, doi: 10.3390/s23052760.
URL: <https://www.mdpi.com/1424-8220/23/5/2760>
- [2] Y. Zhauniarovich, I. Khalil, T. Yu, and M. Dacier, "A Survey on Malicious Domains Detection through DNS Data Analysis," *ACM Computing Surveys*, vol. 51, no. 4, Art. no. 67, pp. 1–36, Sep. 2018, doi: 10.1145/3191329.
URL: <https://dl.acm.org/doi/10.1145/3191329>
- [3] T. Holz, C. Gorecki, K. Rieck, and F. C. Freiling, "Measuring and Detecting Fast-Flux Service Networks," in *Proc. 15th Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, Feb. 2008.
URL: <https://www.ndss-symposium.org/wp-content/uploads/2017/09/Measuring-and-Detecting-Fast-Flux-Service-Networks-paper-Thorsten-Holz.pdf>
- [4] Palo Alto Networks Unit 42, "Fast Flux 101: How Cybercriminals Improve the Resilience of Their Infrastructure to Evade Detection and Law Enforcement Takedowns," Unit 42 Threat Research, Palo Alto Networks. Accessed: Apr. 22, 2026.
URL: <https://unit42.paloaltonetworks.com/fast-flux-101/>
- [5] MITRE, "Dynamic Resolution: Fast Flux DNS, Sub-technique T1568.001," MITRE ATT&CK Enterprise Matrix. Accessed: Apr. 22, 2026.
URL: <https://attack.mitre.org/techniques/T1568/001/>
- [6] MITRE, "Dynamic Resolution: Domain Generation Algorithms, Sub-technique T1568.002," MITRE ATT&CK Enterprise Matrix. Accessed: Apr. 22, 2026.
URL: <https://attack.mitre.org/techniques/T1568/002/>
- [7] A. D. Wong, "Detecting Domain-Generation Algorithm (DGA) Based Fully-Qualified Domain Names (FQDNs) with Shannon Entropy," *arXiv:2304.07943 [cs.CR]*, Apr. 2023.
URL: <https://arxiv.org/abs/2304.07943>
- [8] C. Hajaj, N. Hason, and A. Dvir, "Less Is More: Robust and Novel Features for Malicious Domain Detection," *Electronics*, vol. 11, no. 6, Art. no. 969, Mar. 2022, doi: 10.3390/electronics11060969.
URL: <https://www.mdpi.com/2079-9292/11/6/969>
- [9] S. Schüppen, D. Teubert, P. Herrmann, and U. Meyer, "FANCI: Feature-based Automated NXDomain Classification and Intelligence," in *Proc. 27th USENIX Security Symp.*, Baltimore, MD, USA, Aug. 2018, pp. 1165–1181.
URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/schuppen>
- [10] The Zeek Project, "Book of Zeek," Zeek documentation. Accessed: Apr. 22, 2026.
URL: <https://docs.zeek.org/>

- [11] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, "Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation," in Proc. 26th Network and Distributed System Security Symp. (NDSS), San Diego, CA, USA, Feb. 2019.
URL: <https://www.ndss-symposium.org/ndss-paper/tranco-a-research-oriented-top-sites-ranking-hardened-against-manipulation/>
- [12] Q. Scheitle et al., "A Long Way to the Top: Significance, Structure, and Stability of Internet Top Lists," in Proc. ACM Internet Measurement Conf. (IMC), Boston, MA, USA, Oct. 2018, pp. 478–493, doi: 10.1145/3278532.3278574.
URL: <https://dl.acm.org/doi/10.1145/3278532.3278574>
- [13] S. Torabi, A. Boukhtouta, C. Assi, and M. Debbabi, "Detecting Internet Abuse by Analysing Passive DNS Traffic: A Survey of Implemented Systems," IEEE Communications Surveys & Tutorials, vol. 20, no. 4, pp. 3389–3415, 2018, doi: 10.1109/COMST.2018.2849614.
URL: <https://ieeexplore.ieee.org/document/8391393>
- [14] Active Countermeasures, "Malware of the Day: dnscat2 DNS Tunneling," Active Countermeasures. Accessed: Apr. 24, 2026.
URL: <https://www.activecountermeasures.com/malware-of-the-day-dnscat2-dns-tunneling/>
- [15] R. Bowes, "dnscat2," GitHub. Accessed: Apr. 24, 2026.
URL: <https://github.com/iagox86/dnscat2>

11. Appendices

Appendix A – GitHub repository and access description:

The DNSWatch source code is publicly available on GitHub at:

<https://github.com/bar1socl/DNSWatch>

The repository contains the two Python modules: main.py for DNS extraction and detect.py for detection and scoring, alongside with a README.md file providing full installation and usage instructions. To run DNSWatch, Python 3.12 and tshark must be installed on the host machine. Required Python libraries can be installed using the following command:

```
pip install pandas tldextract matplotlib
```

Full step-by-step usage instructions are provided in the README.md file within the repository.

Appendix B — Evidence of Project Management and Supervision

The following screenshots provide evidence of ongoing supervisor engagement throughout the project. Key design decisions including the redesign of the scoring engine, adoption of the Tranco popularity list, and consolidation of the dashboard output were all discussed and influenced by supervisor feedback.

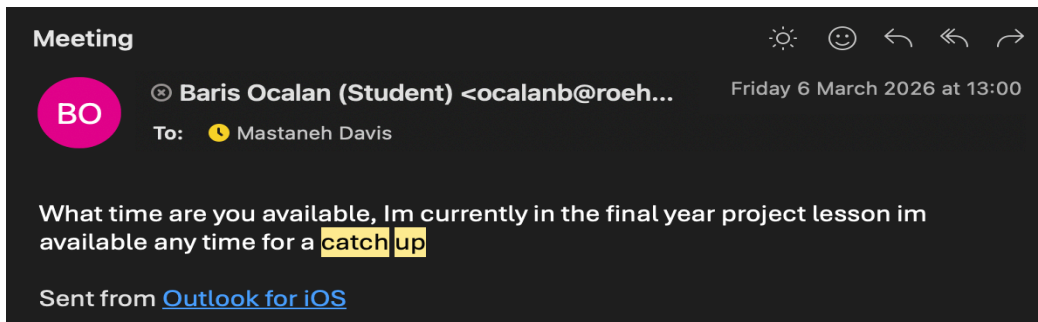


Figure 7 Mastaneh Catch up Example #1

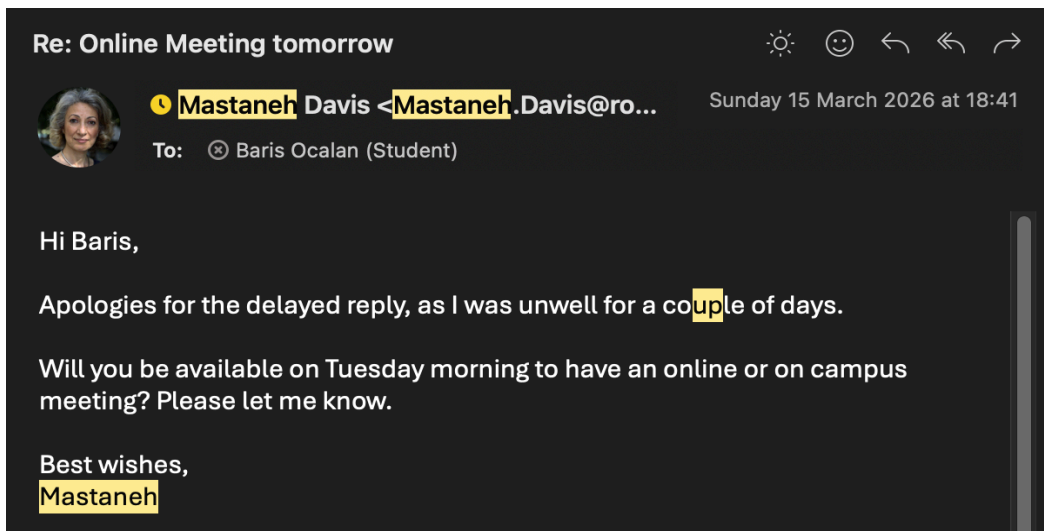


Figure 8 Mastaneh Catch up Example #2

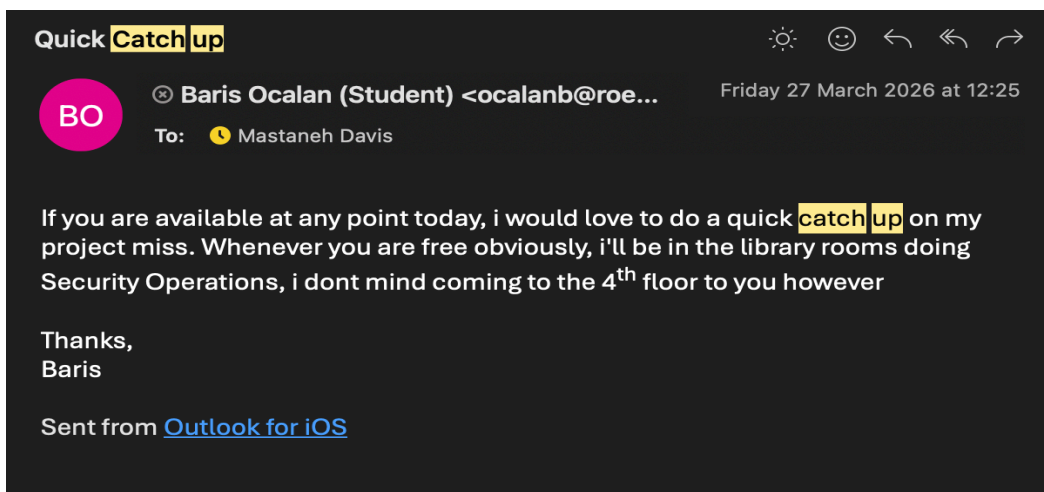


Figure 9 Mastaneh Catch up Example #3

Appendix C – Ethics Form

The approved ethics form is below:

94 29/04/2026 15:01 29/04/2026 15:13
ocalanb@roehampton.ac.uk Baris Ocalan Baris Ocalan
OCA23578960 Cyber Security Undergraduate Final Year Project (CMP-X301-0)
3 29/04/2026 DNSWatch DNS traffic is a common vector for malware communication and data exfiltration, yet manual DNS analysis in tools like Wireshark is slow and requires significant expertise. This project builds DNSWatch — a Python tool that reads packet capture files, applies a multi-signal scoring model to detect suspicious DNS behaviour, and produces structured reports. The research reviews existing detection techniques including Shannon entropy, TTL analysis, and NXDOMAIN patterns to inform the scoring design. No If my plans change and humans become involved in my research in any capacity, I submit a new form; I confirm that the details supplied on this form is correct and my research complies with ethical standards

Supervisor: Dr Mastaneh Davis
Dr Mastaneh Davis PhD, SFHEA, MIET, FIMA
Associate Professor in Cyber Security,
Programme Leader BSc(Hons) Cyber Security
Head of Learning and Teaching for Computing
School of Computing, Engineering and the Built Environment

Figure 10 Ethics form Approval

Appendix D – Risk Register

Below is the risk register:

Table 6 Risk Register Appendix D

Risk	Likelihood	Impact	Mitigation
Loss of project files due to hardware failure	Medium	High	All project files stored in OneDrive with automatic cloud synchronisation across two devices (Windows PC and MacBook Air)
Tshark parsing errors producing incorrect DNS data	Medium	High	Extraction output was manually validated against Wireshark's own GUI to confirm field accuracy before any

			detection logic was applied
High false positive rate reducing the usefulness of detection output	High	High	Identified early through testing on real traffic. Addressed through iterative threshold tuning, multi-signal correlation requirement, and Tranco-based trust layer
Dependency issues preventing the tool from running on different operating systems	Medium	Medium	Cross-platform compatibility was built into main.py using automatic OS detection for the tshark path. Tested on both Windows and Mac
Scope creep delaying completion of the artefact	Medium	High	Scope was managed through regular supervisor meetings. Features such as MISP integration and DNS over HTTPS analysis were identified as valuable but deferred to future work rather than risking incomplete implementation
Inability to test against real malicious traffic	Medium	High	A publicly available dnscat2 DNS tunnelling dataset was sourced to provide a controlled malicious traffic sample for evaluation

Appendix E – Professional and Peer Support Badges

Below is my Professional and Peer Support Badges that I have accumulated during my final year project studies:



Professional
practice badge



Peer supporter
badge

Figure 11 Professional and Peer Supporter Badges